

University of Minho – Department of Informatics
2nd Semester, 2024/2025
Master's Degree in Informatics Engineering
Network Management & Security

Practical Assignment
Sensing with L-SNMPvS
Version 2.0

Main Goal:

- Consolidation of knowledge about the protocols, mechanisms, and philosophies of the Internet-standard Network Management Framework (INMF) architecture, with special emphasis on protocol interaction aspects, management object conceptualization, and MIB specification.

Notes:

- The work should be carried out over the course of 50 to 60 effective hours of individual work.

Recommended Requirements:

- Access to a system with at least one freeware package installed that supports SNMP (version 2 or higher), such as Net-SNMP, CMU-SNMP, SCOTTY, etc.
- Use of programming APIs that facilitate the implementation of UDP communication primitives.

WARNING:

- Any violation of software copyrights or intellectual property rights will not be tolerated.

References & Support Material

Support Material:

- *net-snmp* tutorials
- MIBs in `/usr/share/snmp/mibs`;
- <http://net-snmp.sourceforge.net/wiki/index.php/Tutorials/>;
- <http://www.simpleweb.org/>;
- <http://www.snmplinks.org/>.

References:

- M. Rose, *The Simple Book*, Second Edition, Prentice Hall, 1996.
- W. Stallings, *SNMP, SNMPv2, SNMPv3, and RMON 1 and 2*, Addison-Wesley, 2000.
- D. Mauro, K. Schmidt, *Essential SNMP*, O'Reilly, 2001.
- Refer to additional resources in the "Content" section on the UC's Blackboard.

L-SNMPvS Management Model

In the past, several monitoring and control models have been defined for sensor networks, home automation systems, IoT, etc., based on the SNMP architecture, with specific MIBs designed for the unique functionalities of these systems. In other words, the traditional management strategy applied to network equipment, services, and applications—particularly in the Internet context—has also been tested in these more specialized systems. However, it was never truly adopted by the industry, which has consistently favored solutions functionally closer to the low-level technologies embedded in the devices being managed. The industry prioritized ease of implementation over interoperability, advanced monitoring and control capabilities, and user interface sophistication. As a result, SNMP-based management, despite its proven adaptability, has largely been sidelined in favor of proprietary or device-centric approaches.

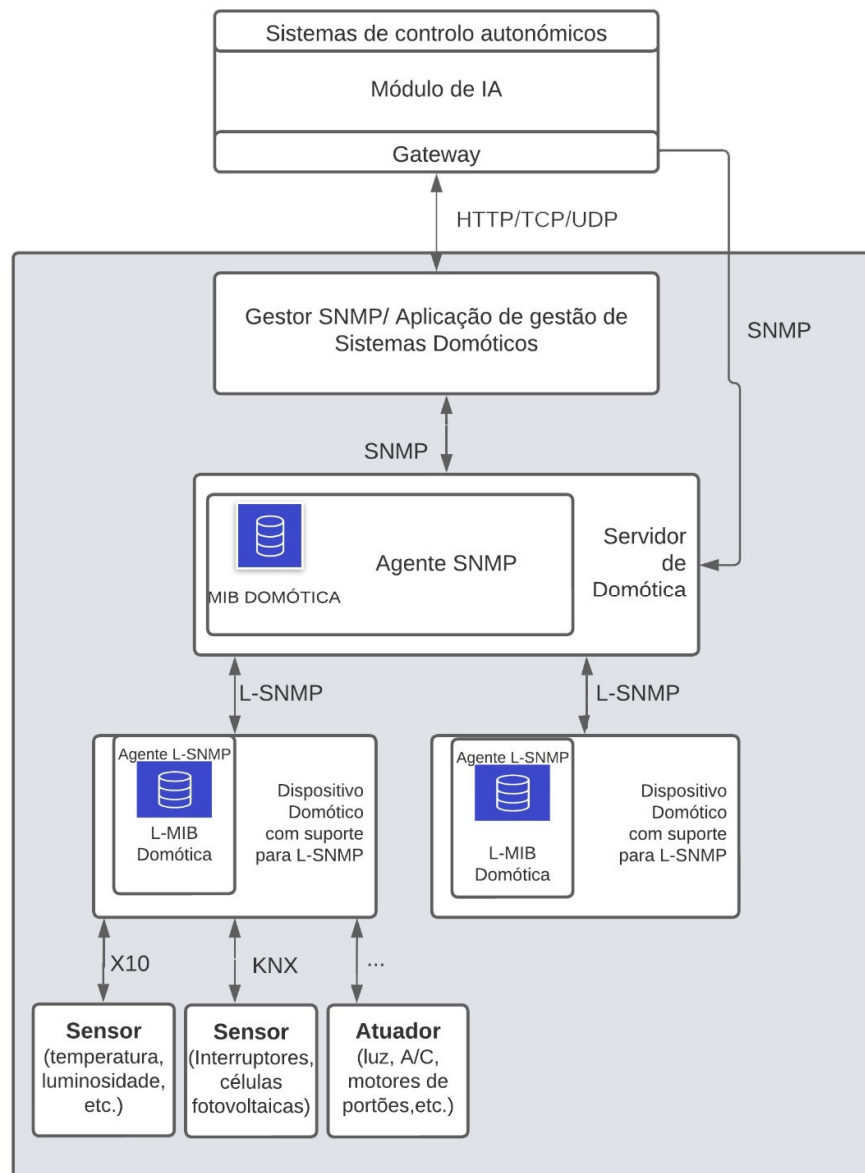


Figure 1: Management Architecture for Home Automation Systems Using SNMP and L-SNMP.

In a recent experimental project developed at the University of Minho in collaboration with DTX Lab, researchers have created an innovative model to enhance Internet-based management protocols. This

solution maintains compatibility with traditional SNMP agents while introducing a complementary architecture called Light Secure SNMP (L-SNMPvS), which implements specialized L-SNMPvS management agents. Key features of this architecture include a Lightweight MIB Implementation, that is, while following SNMP paradigms, agents deploy special Light MIBs (L-MIBs) directly on smart home devices or sensing hub devices, significantly reducing computational requirements for device manufacturers compared to traditional MIBs. Furthermore, the new model incorporates additional sensing and control capabilities beyond standard SNMP functionality and may function as an intermediate functional layer between hardware-based sensor devices and conventional SNMP agents/control systems, although a native mode with direct L-SNMPvS communication between all entities of a management system, from sensor devices up to the UI module, is possible (completely independent of SNMP). This represents a significant advancement in network management for resource-constrained environments, offering manufacturers a more efficient alternative to traditional SNMP implementations while expanding functionality.

In summary, the L-SNMPvS model is particularly suitable for management systems when an additional abstraction layer is beneficial for defining resource functionalities and where systems face computational or communication constraints. It's ideal application scenarios include in-vehicle local systems, sensor networks, IoT ecosystems and home automation. L-SNMPvS emerges as a viable alternative to standardized solutions like CoaAP, MQTT or Matter-based protocols, offering lightweight operation for constrained devices, backward compatibility with SNMP paradigms and enhanced sensing functionalities beyond traditional SNMP.

Assignment Goals

The main objective of this work is the implementation of a high-performance monitoring system supporting virtual sensors and using the L-SNMPvS model. The system should be simple, without requiring a connection to an SNMP system or an artificial intelligence system for control and monitoring. Communication between the agent and the manager should be carried out using a secure version of the L-SNMP protocol, L-SNMPvS, as specified in this statement.

It should consist of the following software modules:

- **Monitoring Console** – Allows the user to check real-time values (and their temporal evolution) of virtual sensors and define some monitoring parameters (sampling rate, active sensor groups, etc.). This console, which serves as a management application, should include a simple interface, preferably secure (authenticated and confidential), for a human user.
- **Virtual Sensors**: Implements a set of simple sensors, configurable in terms of maximum and minimum values, virtual value generation function for sensor readings, virtual generation rate of sensor readings, value precision, etc.
- **L-SNMPvS Agent**: Implements an L-MIB for sensor network monitoring and provides functional connection instrumentation with virtual equipment modules (it can even be implemented as a single module). It is recommended that the L-MIB implementation follows the definition included in the respective section of this document.

L-SNMP PDU (Non-Secured)

The L-SNMPvS protocol is based on SNMP. It is simpler but allows for some additional functionalities that SNMP does not directly support (such as "beacons" and "broadcast monitoring"). For this project, a simplified implementation of L-SNMPvS is required, following the specifications outlined below, with encapsulation in UDP datagrams. The core L-SNMP content of a L-SNMPvS PDU shall be identified as the L-SNMP PDU, while the entire data stream used for the secured version shall be identified as the L-SNMPvS Message.

The core L-SNMP PDU of the L-SNMPvS protocol messages has the following format:

Tag | Type | Timestamp | MSG-ID | IID-List | V-List | T-List | E-List

The specification of each field in the L-SNMP message is as follows:

- **Tag** – A fixed and predefined value, the same for all PDUs, serving as a quick protocol identifier to simplify an initial PDU verification. The value must be encoded as a short eight-byte sequence as defined by the L-SNMPvS model;
- **Type** – Indicates the message type and can have one of four possible values: *get-request*(0) and *set-request*(1), which are used by managers/clients, and *notification*(2) and *response*(3), which are used by agents/servers. The value is encoded as a single byte, as defined by the L-SNMPvS model;
- **Timestamp** – A timestamp with millisecond precision, indicating the exact moment (date, hours, minutes, seconds, and milliseconds) when the request was sent by the manager/client (for get-request or set-request messages) or when the response or notification was sent by the agent/server. The value is encoded as a timestamp type defined in the L-SNMPvS model, comprising seven fields: `day:month:year:hours:mins:secs:ms` (where ms represents milliseconds);
- **MSG-ID** – A randomly generated value that uniquely identifies a request from a manager so that the agent can match the corresponding response, or that uniquely identifies notification messages from the agent. It is encoded as a short eight-byte sequence, as defined in the L-SNMPvS model;
- **IID-List** – A list of object instance identifiers. Each element in the list is an Instance Identifier (IID), which is analogous to SNMP OIDs. A more detailed explanation can be found in the dedicated section of this document. Each IID must be encoded following the encoding rules for IIDs in the L-SNMPvS model;
- **V-List** – A list of values corresponding to the previous IID list (i.e., the first value corresponds to the instance referred to by the first IID, and so on). Each value must be encoded using the data type encoding rules specified in the L-SNMPvS model;
- **T-List** – A list of timestamps, with millisecond precision, indicating the time elapsed since the last reset/reboot/power-on of the agent until each respective instance value was recorded/updated in the agent's L-MIBvS (i.e., indicating the age of each instance value included in the message). The value is encoded as a timestamp type defined in the L-SNMPvS model and consists of four fields: `hours:mins:secs:ms`. This list contains zero elements in messages sent by managers;
- **E-List** – A list of errors encountered by the agent/server. The manager does not use this field and should send an empty list. The agent, however, should include error codes defined for L-SNMPvS. For this project, students should define a small set of possible error codes, but at a minimum, the following are suggested: 0 (no errors), 1 (message decoding error), 2 (Tag error), 3 (unknown message type), 4 (duplicate message), 5 (invalid or unknown IID), 6 (unknown value type), 7 (unsupported value), 8 (value list does not match IID list).

The fields in the L-SNMPvS PDU that are lists (IID, timestamps, values, and errors) must be encoded as a list containing zero or more elements (fields marked with * are optional):

N-Elements | 1st-Element* | 2nd-Element* | ... | Nth-Element*

The first field is mandatory and indicates the number of elements in the list (the minimum value is zero, meaning the list is empty and contains no elements). Starting from the second field, which only appears if the list contains one or more elements, the list elements are encoded sequentially, from the first to the nth element. Each list element must be encoded following the rules defined in the L-SNMPvS model.

Messages of the *get-request* and *set-request* types are sent by the manager application to obtain information (values of object instances) implemented by the agent's instrumentation.

The *response* message can only be used by agents to send the information requested in the corresponding *get-request* or *set-request* sent by the manager. Furthermore, in the case of a *set-request*, the agent must attempt to modify the instances referred to in the request with the respective values also present in the request before responding to the manager (which will require the implementation of control/configuration procedures for the resources abstracted by the objects in question).

Like SNMP, the L-SNMPvS protocol is asynchronous, asymmetric, atomic, and unconfirmed (there is no obligation to respond, no confirmation-only responses, and no predefined timeout values).

Finally, *notification* messages are used by agents to send status information (values of special object instances that have been pre-defined for this type of notification) to all managers on the local network via broadcasting, or to a specific manager. This type of message is also used for agents to implement a *beacon* transmission service from the agent to the local network via broadcasting (the beacons contain generic information about the L-SNMPvS equipment).

L-SNMPvS Message

A secured version of the L-SNMPvS protocol should include the needed security mechanisms and technologies so the following security guarantees can be implemented:

- Weak mutual authentication – this feature should allow agents and managers to authenticate each other using, at least, a security tuple ($K_A; K_M$) of symmetric keys/secrets for each combination of agent/manager (these symmetric keys/secrets must be known in advance by the entities using a simple but secure method like an encrypted configuration file), i.e., each manager must have an associated K_M key/secret known only by itself and the agent, and the agent must have its own K_A key/secret known only by itself and the manager associated to the K_M key/secret.
- Strong confidentiality – this feature should guarantee that only the entities associated with a security tuple can extract and understand the content of the core L-SNMP PDU inside the L-SNMPvS Message; at least, a strong encryption mechanism using the security tuple ($K_A; K_M$) should be implemented to encrypt the core L-SNMP PDU inside the L-SNMPvS Message.
- Strong verification of content tampering – this feature should guarantee that the receptor of the L-SNMPvS Message can verify with great confidence that the entire content of the message was not tampered while in transit; to implement this security guarantee a secure summary (or hashing) mechanism should be implemented.

In the scope of this project no further security guarantees should be implemented.

The Message of the L-SNMPvS protocol has the following format:

Sender-ID | Receiver-ID | Sec-Model-ID | Sec.-Data-Size | Sec.-Data*

The specification of each field in the L-SNMPvS Message is as follows:

- **Sender-ID** – A predefined value identifying the sender of the message (agent or manager); although not being the safest approach, in this project, each entity shall use only one identifier; it is encoded as a short eight-byte sequence, as defined in the L-SNMPvS model;
- **Receiver-ID** – A predefined value identifying the intended recipient of the message (agent or manager); although not being the safest approach, in this project, each entity shall use only one identifier; it is encoded as a short eight-byte sequence, as defined in the L-SNMPvS model;

for broadcasting interactions, as in beacons, the receiver identification shall be equal to a short eight-byte sequence with each byte equal to zero;

- **Sec.Model-ID** – The identification of the security model that should be used by both sender and receiver; this will determine which security mechanisms shall be implemented; for the requisites and security guarantees defined for this project a value of 128 shall be used, taking into consideration that this field is encoded as a single byte, as defined by the L-SNMPvS model; for broadcasts interactions, where no security guarantees are implemented, the security model identifier shall be equal to zero (identifying the security model as “None”);
- **Sec.-Data-Size** – This field indicates what’s the size, in bytes, of the **Sec.-Data** field and it is encoded as an integer of 32 bits, as defined in the L-SNMPvS model;
- **Sec.-Data** – This is the resulting sequence of bytes of applying all security mechanisms to the core L-SNMP PDU and adding all necessary auxiliary data, like hash/checksum values; it is encoded as long byte sequence, as defined in the L-SNMPvS model; when the security model is identified as “None”, then this is the exact L-SNMP PDU byte sequence.

It should be noted that there should be no security measures applied to the beacons sent by the agent to the managers using broadcast communications. That is, when the agent broadcasts beacons to the local network or sends them to a group of specific managers of a list previously defined using unicast communication, the L-SNMPvS Message will include a Security Model ID of “None” and the **Sec.-Data** field corresponds to the original L-SNMPvS beacon PDU.

Instance Identifier

All instances of objects implemented in the instrumentation of an L-SNMPvS MIB (L-MIBvS) must be organized in a data structure, which can be of two types: *Group* or *Table*.

Instances are identified/referred to by a sequence of integers known as the Instance Identifier (IID), where each integer serves a specific representative function.

Here is an outline of how the IID is structured, including optional fields (marked with *):

Structure | Object | 1st-Index* | 2nd-Index*

In L-MIBvS, there is no exact equivalent to the Group concept as in traditional MIBs. However, simple objects cannot be directly referenced at the first level of identification. The fields that identify the data structure and the object within that structure are mandatory. Therefore, the sequence of integers that forms an IID (Instance Identifier) must, at a minimum, contain two integers: a Data Structure Identifier and an Object Identifier. The first field identifies the data structure, such as whether the object belongs to a group or table-like structure. It's required to categorize the object instance within a certain organizational context in the MIB. The second field identifies the specific object within the structure. It could refer to a sensor, device parameter, or any other element that needs to be monitored or controlled within the system. These two fields form the minimum structure for an IID, ensuring that object instances are properly organized and referenced within the L-MIBvS framework.

The encoding of an IID value follows the rules defined for L-SNMPvS and will be detailed in later sections of this document. However, the meaning of each field in an IID is as follows:

- **Structure** – This field identifies the highest-level data structures in the L-MIBvS. Its value must be greater than zero. A structure can be a *Group* or a *Table*.
- **Object** – This field identifies one of the objects that belongs to the data structure referenced by **Structure**. In groups there is no inherent relationship between the objects other than possibly a generic functional or administrative grouping. In other words, a group should contain a set of

objects that are related functionally or administratively in a general manner, but there should be no operational relationship in the value of their instances. Objects in a group or a table in L-MIBvS are simple objects. There is no concept of a list of objects (potentially of different types) as in SNMP. Instead, all objects can have more than one instance, which functions as a sequence of instances of a single object.

- A *Table* is defined as a set of columns, where each column is a set of instances of the same object type (all of the same kind). Additionally, there is an operational relationship between the instances of the objects. Instances are identified by a reference order (indexes), and instances with the same reference number (index) in all objects are operationally related, that is, the order of the instances identifies a collection of related object instances about the same resource, managed in a way similar to how MIB tables use indexes as keys. Further, while objects in a Group can have a different number of instances implemented on the same agent, the objects/columns in a Table must have the same number of instances implemented on the same agent.
- A key in an SNMP OID identifies a row in a MIB table. In an L-MIBvS, the values of one or two indexes represent one or more rows of a table.
- When the value of *Object* is zero, the IID represents the number of objects in the group or table (in the case of tables, this refers to the number of columns in the table).
- When the IID does not contain any indexes, and the value of *Object* is greater than zero, the IID implicitly represents the first instance of the indicated object. This is equivalent to *Group.Object.1*.
- The N instances of an object are identified by indexes 1 to N, respectively. If there is only the first index value and it is zero, the IID represents the number of instances of the object currently implemented by the agent (for tables, this refers to the total number of instances in the respective column, i.e., the number of rows in the table). If there is only the first index value and it is greater than zero, the IID identifies one specific instance. If the index value is greater than the total number N of instances implemented by the agent, the IID should be considered invalid by the agent. If there are two indexes and both are zero, the IID represents the set of values of all instances of that object (for tables, it represents the set of values of all instances in a column). If there are two indexes and both are greater than zero, the IID represents the set of values of the instances identified by the indexes between the first and second indexes in the IID. If there are two indexes and only one is zero, or if the second is smaller than the first, or if either index is greater than N, the IID should be considered invalid by the agent.

It should be noted that in the L-SNMPvS model, an agent responding on a UDP service port can only implement one specific L-MIBvS. Therefore, the identification of the L-MIBvS that the agent supports is implicit in all IIDs used in the PDU between the managers and the agent. However, in the object group dedicated to the generic equipment and the software module of the agent itself, there should always be one or more simple objects that allow identification of the L-MIBvS implemented by the agent. Furthermore, at least one instance value of one of these objects must be included in the beacons sent by the agent to the local network. This ensures that the agents' L-MIBvS can be identified and referenced, making the system compatible with the specific MIB structure and allowing other network devices to recognize the agent's capabilities.

Types of Values and their Encodings

In the L-SNMPvS model, there is a basic data value type with no associated semantics: *byte*. For this data type, a direct binary encoding format is defined. All other types of object instance values in the L-SNMPvS model have some additional semantics and are encoded indirectly or using variations of this basic type. Therefore, besides the byte type, the model allows the definition of object instance values of

four additional types: *integer*, *timestamp*, *string*, and *IID*. Additionally, sequences of values of the same type can be defined for some of the aforementioned types.

The encoding of the values of the instances of objects follows this sequence of fields (each field has a size of multiple of eight bits; fields marked with * are optional):

Data-Type | N-Bytes* | Value-1 | Value-2* | ... | Value-N*

The Data-Type field must be encoded as eight bits and defines the value type:

- A value equal to **00000000** is used to encode instances of the *byte* type; in this case, the N-Bytes field does not exist, only the Value-1 field of eight bits exists, i.e., it is used to encode the value of a single byte (value from 0 to 255);
- When the Data-Type field value is **00000001**, it defines a *byte* short sequence; in this case the N-Bytes field, which occupies eight bits, indicates how many bytes follow in the byte array (maximum size of 255 bytes). The byte values in the array are in Value-1 to Value-N (eight bits each);
- When the Data-Type field value is **00000010**, it defines a *byte* long sequence; in this case, there is the N-Bytes field, which occupies sixteen bits, indicating how many bytes follow in the byte array (maximum size of 65535 bytes). The byte values in the array are in Value-1 to Value-N (eight bits each);
- When the Data-Type field value is **000001XX**, it encodes a value of an *integer* instance, and the last two bits indicate the size of the single Value-1 field (XX is **00** for eight bits, **01** for sixteen bits, **10** for thirty-two bits, and **11** for sixty-four bits). In this case, there is no N-Bytes field. The value of Value-1 is the integer value of the instance represented in two's complement;
- When the Data-Type field value is **000010XX**, it encodes a short sequence of *integer* values, and the last two bits indicate the size of the Value-1 to Value-N fields (XX is **00** for eight bits, **01** for sixteen bits, **10** for thirty-two bits, and **11** for sixty-four bits), encoded in two's complement. In this case, the N-Bytes field occupies eight bits, indicating the size of the sequence (maximum size of 255 integers), i.e., the number of Value-1 to Value-N fields;
- When the Data-Type field value is **000011XX**, it encodes a long sequence of *integer* values, and the last two bits indicate the size of the Value-1 to Value-N fields (XX is **00** for eight bits, **01** for sixteen bits, **10** for thirty-two bits, and **11** for sixty-four bits), encoded in two's complement. In this case, the N-Bytes field occupies sixteen bits, indicating the size of the sequence (maximum size of 65535 integers), i.e., the number of Value-1 to Value-N fields;
- When the Data-Type field value is **0001000X**, it encodes a value of a *timestamp* instance, and the last bit indicates the type of timestamp (**0** for a timestamp of the form `day:month:year:hours:mins:secs:ms` and **1** for a timestamp of the form `days:hours:mins:secs:ms`). In these cases, there is no N-Bytes field. The values for seconds and milliseconds are encoded as a single value in milliseconds in Value-1 (a binary value of sixteen bits between 0 and 59999). The values for hours and minutes are encoded as a single value in minutes in Value-2 (a binary value of sixteen bits between 0 and 14399). The values for days in the second type of the timestamp type are encoded in Value-3 (a binary value of sixteen bits between 0 and 65535). In the first timestamp type, the value stored in Value-3 indicates a specific date, represented as a single integer value equal to $(year - 2000) * 2^8 + month * 2^4 + day$, and encoded in a binary value of sixteen bits;
- When the Data-Type field value is **0010XXXX**, it encodes a value of an instance of a string type, and the last 4 bits indicate the encoding to be used to interpret the character sequence: **0000** indicates encoding using the ASCII normalized table with 7 bits per character, **0001**

indicates encoding using the extended ASCII table or ISO-8859-1 with 8 bits per character, etc. The N-Bytes field occupies sixteen bits, indicating the size of the sequence (maximum size of 65535 characters), i.e., the number of Value-1 to Value-N fields (each occupying eight bits for ASCII normalized or extended encoding);

- When the Data-Type field value is 010000XX, it encodes an IID of integer values, and the last two bits indicate whether the IID includes zero, one, or two indexes: XX is 00 if it does not include indexes, XX is 01 if it includes only the first index, XX is 11 if it includes both indexes. In this case, there is no N-Bytes field. If the IID does not include indexes, only Value-1 and Value-2 exist, each represented by a positive integer value encoded directly in eight bits (note that this limits the maximum number of structures in an L-MIBvS to 255 and the maximum number of objects within a structure to 255). If the IID includes the first index, in addition to Value-1 and Value-2, Value-3 is added to contain the index value, encoded as a positive integer directly in sixteen bits (this implies the maximum number of instances per object is 65535). If the IID includes both indexes, in addition to Value-1, Value-2, and Value-3, Value-4 is added to contain the second index value, encoded as a positive integer directly in sixteen bits.

Security in the L-SNMPvS Model

To be defined in version 2.x of this document. It will include the security guarantees proposed by the model, specifically details on authentication requirements, confidentiality, access control, and data integrity verification.

Sensing L-MIBvS

[...]

-- Structures:

-- (1) device Group

-- (2) sensors Table

-- device (1)

-- This group includes objects that represent characteristics of a sensing system.

device OBJECT {

TYPE Group

INCLUDE lMibId, id, type, beaconRate, nSensors, dateAndTime, upTime, opStatus, reset

NOTIFICATION lMibId, id, type, nSensors, dateAndTime, upTime, lastTimeUpdated, opStatus

NOTIFICATION-RATE beaconRate

DESCRIPTION "Simple list of objects, where each object represents a characteristic from a sensing device."

IID 1 }

device.lMibId OBJECT {

TYPE Integer

ACCESS read-only

DESCRIPTION "Number identifying the L-MIB that the agent in the device implements."

IID 1.1 }

device.id OBJECT {

TYPE String

ACCESS read-only

DESCRIPTION "String identifying the device where the agent runs (for example, this could be the MacAddress)."

IID 1.2 }

device.type OBJECT {

```
TYPE String
ACCESS read-only
DESCRIPTION "Text description for the type of device ("Sensing Hub", for example)."
```

IID 1.3 }

```
device.beaconRate OBJECT {
  TYPE Integer
  ACCESS read-write
  DESCRIPTION "Frequency rate in seconds (for example, one beacon at each five seconds) for
  issuing a notification message with information from this group that acts as a beacon
  broadcasting message to all the managers in the LAN. Minimum value is one second, that is,
  one beacon will be sent at every second. If this instance value is set to zero the beacons
  are halted."
```

IID 1.4 }

```
device.nSensors OBJECT {
  TYPE Integer
  ACCESS read-only
  DESCRIPTION "Number of sensors implemented/connected in/to the device and represented in
  the sensors Table."
```

IID 1.5 }

```
device.dateAndTime OBJECT {
  TYPE Timestamp
  ACCESS read-write
  DESCRIPTION "System date and time setup in the device."
```

IID 1.6 }

```
device.upTime OBJECT {
  TYPE Timestamp
  ACCESS read-only
  DESCRIPTION "For how long the device is working since last boot/reset."
```

IID 1.7 }

```
device.opStatus OBJECT {
  TYPE Integer
  ACCESS read-only
  DESCRIPTION "The operational state of the device, where the value 0
  corresponds to a standby operational state, 1 corresponds to a normal operational state
  and 2 or greater corresponds to an non-operational error state."
```

IID 1.8 }

```
device.reset OBJECT {
  TYPE Integer
  ACCESS read-write
  DESCRIPTION "Value 0 means no reset and value 1 means a reset procedure of the device
  (including the agent software) must be done."
```

IID 1.9 }

```
-- sensors (2)
-- This Table includes objects that permit access to all sampled values from all the
-- sensors implemented/connected to the device

sensors OBJECT {
  TYPE Table
  INCLUDE id, type, sampleValue, minValue, maxValue, lastSamplingTime, samplingRate
  NOTIFICATION id, sampleValue, lastSamplingTime
  NOTIFICATION-RATE samplingRate
  DESCRIPTION "Table with information for all sensors implemented/connected to the device."
  IID 2 }

sensors.id OBJECT {
  TYPE String
  ACCESS read-only
  DESCRIPTION "Tag identifying the sensor (the MacAddress, for example).\"
  IID 2.1 }

sensors.type OBJECT {
  TYPE String
  ACCESS read-only
  DESCRIPTION "Text description for the type of sensor ("Light", for example).\"
  IID 2.2 }

sensors.sampleValue OBJECT {
  TYPE Integer
  ACCESS read-only
  DESCRIPTION "Last value sampled by the sensor in percentage of the interval between
  minValue and maxValue.\"
  IID 2.3 }

sensors.minValue OBJECT {
  TYPE Integer
  ACCESS read-only
  DESCRIPTION "Minimum value possible for the sampling values of the sensor.\"
  IID 2.4 }

sensors.maxValue OBJECT {
  TYPE Integer
  ACCESS read-only
  DESCRIPTION "Maximum value possible for the sampling values of the sensor.\"
  IID 2.5 }

sensors.lastSamplingTime OBJECT {
  TYPE Timestamp
  ACCESS read-only
  DESCRIPTION "Time elapsed since agent's last reboot/reset for the last sample obtained by
  the sensor.\"
  IID 2.6 }

sensors.samplingRate OBJECT {
  TYPE Integer
  ACCESS read-write
  DESCRIPTION "Frequency rate in Hertz for issuing notifications messages with information
  from each sensor to all managers in the LAN using broadcast. If value is set to zero the
  notifications for this sensor are halted. By default, the sampling rate for all sensors
  should be ten Herzt.\"
  IID 2.7 }
```

Report and Other Recommendations

The code for the agent and manager prototypes can be developed in any programming language of choice. APIs, functions, or third-party code snippets may be used to implement technological aspects, provided they are properly referenced. In case of doubt, consult the instructor to verify the appropriateness of their use in the context of the assignment.

The code must be clear and follow naming conventions for variables, types, functions, and constants. It should be structured in the most modular way possible, avoiding unnecessary complexity and enabling reuse whenever possible. The quality and correctness of the code are not measured by its size! Students should document/explain the code through comments in relevant sections and in the report. All code files must include a header with relevant information identifying the authors and explaining the main functions/classes created, data types, and variables used.

The report can be written in any format and editor that is most convenient and must include, at a minimum:

- A title page with the project title and author identification (including a photo), university, course, subject, and submission date;
- A table of contents;
- A section discussing the chosen strategies, decisions made, adopted mechanisms and technologies, including any optimizations;
- A section with an explanation and critical analysis of the main functions/classes implemented, their key strengths, and their most important limitations;
- A conclusion section, including a discussion about what could have been improved or added but was not achieved;
- A list of any references, scientific articles, or informal online resources that were useful.

The report is intended for evaluation by the instructor, so avoid including generic or irrelevant information that the instructor already knows. Be concise and clear. Whenever making an important statement in the report that is authored by a third party, directly based on third-party claims, or derived from external sources, proper referencing must be provided, and the sources must be listed in the references section.

The report may include relevant parts of the code when they help with analysis and justification. Do not include unnecessary code in the report text. Whenever possible, comment directly in the code instead. If including code snippets, diagrams, or images, keep in mind that others may need to print the report for reading and analysis. Avoid dark background colors. Be environmentally friendly and help save toner/ink.

The submitted material should include only two files: a PDF file with the report and a ZIP file containing all project code files within a directory named `GSR-24_25-Student_Number.zip`, e.g., `GSR-24_25-83974.zip`.

Finally, during the project defense, it is recommended to answer honestly and concisely only the questions asked, to ensure that presentation sessions do not extend significantly beyond 30-40 minutes.